

Requirements Document

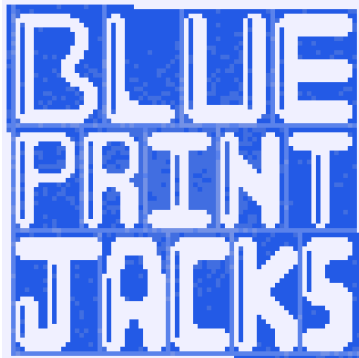
November 26, 2025 – Version 2.0

PROJECT SPONSOR



- ❖ **Chris Ortiz** Senior Technologist, Tools & Infrastructure
- ❖ **Rex Jackson** Vice President, Tools & Infrastructure

TEAM DETAILS



- ❖ Team Name: **Blueprint Jacks**
- ❖ Team's Faculty Mentor: **Ogonna Eli**
- ❖ Team Members: **Jackson Belzer, Christian Lamb, Juan José Serrano Mora, Alejandro Benito Marcos**

Accepted as baseline requirements for the project

Client Signature

Jackson Belzer
Team Lead Signature

Date

11/25/2025
Date

Table of Contents

1. Introduction.....	2
2. Problem Statement.....	3
3. Solution Vision.....	5
4. Project Requirements.....	7
5. Potential Risks.....	19
6. Project Plan.....	22
7. Conclusion.....	24
8. Glossary and Appendices.....	25

1. Introduction

1.1. Industry Context

The global data storage industry is a key pillar of the modern digital economy. As the world produces in excess of 120 zettabytes of data each year, the ongoing demand for trusted and high-performance storage solutions will only continue to grow in every sector, whether this be consumer electronics, cloud computing, enterprise servers or artificial intelligence systems. Demand for the storage device market alone was worth more than \$80 billion in 2024, and it is expected to continue to grow as connected devices, edge computing, and IoT infrastructures expand around the world. The competitive and innovation-driven nature of this ecosystem demands that customers have trusted and reliable product solutions and confidence that their data will be maintained with integrity.

1.2. Sponsor Overview

Our project sponsor, SanDisk, is one of the leading developers and manufacturers of flash memory storage products in the world. As a key player in the global storage industry, SanDisk's technologies form the backbone of many of today's data-driven products and services. There are a variety of products in SanDisk's portfolio, including solid-state drives (SSDs), USB flash drives, memory cards, and embedded storage that are integrated into millions of devices around the world. The company generates value from consumer purchases and from its OEM partnerships, supplying storage components for smartphones, cameras, computers, and industrial systems. SanDisk has a global outreach and places high value on innovation and manufacturing excellence; therefore its business model is based on producing the highest quality and performance in its engineering and design.

2. Problem Statement

2.1. The Problem

While SanDisk has introduced TLA+ and PlusCal to formally verify designs mathematically, most engineers and reviewers still rely heavily on manual visual documentation to understand system behavior. The lack of integration between formal verification tools and diagram/documentation workflows has created a significant productivity and communication gap within the company.

The fundamental challenge is a documentation process that is slow, error-prone, and detached from well-tested models, which raises the possibility that diagrams and design documents do not adequately represent the formally verified logic of the system. Consequently, design errors can propagate through review without notice.

2.2. Current Workflow

At SanDisk, the firmware engineering teams are responsible for designing, verifying, and documenting the internal logic of complex storage systems. The design of each product begins with a specification of the design concept in plain English, tables and pseudocode, as well as UML diagrams. These design documents are iteratively reviewed by engineers, quality specialists and product managers before a design is approved for implementation.

The workflow can be summarized as follows:

1. **Design Authoring:** Engineers write the design specifications in English, including pseudocode to describe how the system is expected to behave.
2. **Diagram Creation:** Visual documents in the form of flow charts, sequence diagrams, and activity diagrams are created manually to communicate the logic to cross-functional teams.
3. **Design Review:** Documents and diagrams go through a review process with multiple engineers and stakeholders providing feedback through several revision cycles.
4. **Verification:** Once the design is approved, it is internally tested, though the verification process typically does not include formal mathematical verification.

- 5. Documentation and Distribution:** The final documents are exported to PDF and distributed to teams for historical purposes and for validation by the client.

2.3. Key deficiencies

The current workflow has several critical limitations that reduce efficiency and increase the risk of errors:

- ❖ **Manual and time-consuming diagram generation**, which slows down design review.
- ❖ **Risk of inconsistencies between the manually created diagrams and the intended system logic**, since current workflows are not connected to formal verification tools like TLA+.
- ❖ **Disconnected workflow**, with diagrams and PDFs not automatically reflecting formally verified logic.
- ❖ **Lengthy review cycles**, as reviewers must interpret separate pseudocode and diagrams.

These deficiencies highlight the need for an automated system that generates accurate, synchronized diagrams and PDFs from verified PlusCal specifications, improving efficiency, accuracy, and confidence in SanDisk's design process.

3. Solution Vision

3.1. Overview Statement

P2P2P is a Visual Studio Code–integrated tool that automates documentation generation from verified PlusCal specifications. It converts these models into professional UML diagrams and complete PDF documents with minimal manual effort. This reduces time, improves accuracy, and ensures consistent documentation grounded in formally proven system behavior.

3.2. Key Features

The P2P2P system will provide the following core capabilities to solve the deficiencies identified in SanDisk's current workflow:

- ❖ Automated diagram generation
- ❖ Integrated VSCode extension
- ❖ Verification-first approach
- ❖ Professional PDFoutput and cross-platform compatibility.

3.3. System Data and Workflow

- ❖ **Input:** PlusCal .tla files verified via TLC; VSCode settings for styles and preferences
- ❖ **Output:** UML diagrams (PNG/SVG), PlantUML files, polished PDF for design review and project archives
- ❖ **Processing:** Parsing .tla files, converting PlusCal constructs to PlantUML, compiling into PDF; errors surface during diagram generation

3.4. Impact and Trade-offs

Replaces manual diagramming and PDF creation, reducing errors, shortening review cycles, and preserving knowledge. Trade-offs: PlusCal learning curve, limited PlantUML customization, and maintaining specs in PlusCal.

3.5. Alternative Approaches

Manual diagram tools, generic UML generators, and web-based solutions were rejected due to inefficiency, lack of integration, or security concerns. Monolithic TLA+ integration was deferred to simplify development.

3.6. Broader Impact

P2P2P not only solves SanDisk's documentation issues but also lowers the barrier to adopting TLA+, encourages wider use of formal methods, and enables reusable verified PlusCal design patterns that reduce duplicated engineering effort.

3.7. System Architecture Diagram

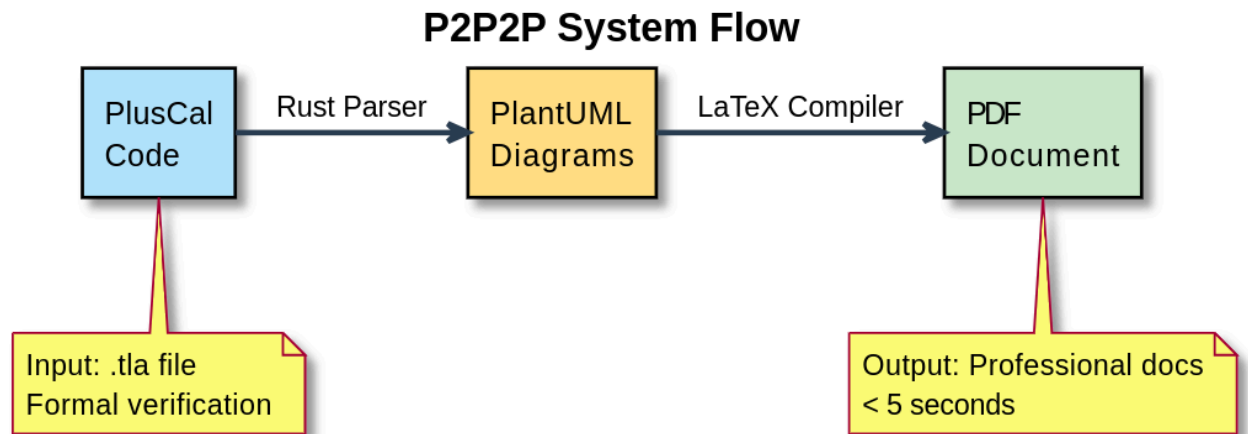


Figure 1: The diagram above illustrates the P2P2P system architecture. The yellow notes are just comments for the items that are pointing to, they are not inputs/outputs.

4. Project Requirements

4.1. Introduction

Having defined the problem, the current workflow together with its deficiencies, and vision for the P2P2P solution established, we can now get to the heart of the document and outline the system's requirements. To recap, we demonstrated the need for an automated, verification-driven documentation pipeline and showed how the proposed solution addresses the challenges in the domain. This section translates that vision into a comprehensive set of requirements that is organized and detailed. We will start from the higher-level domain needs, which broadly summarize what the engineers/stakeholders articulate they need from the system to specific detail in functional, non-functional, and environmental requirements to articulate the behaviors, constraints, and performance expectations such that the needs identified can be achieved.

4.2. Domain-Level Requirements (DLRs)

- ❖ **DLR1:** The system **shall** automatically transform PlusCal models into UML diagrams.
- ❖ **DLR2:** The system **shall** generate professional, consistent PDF documents containing diagrams and verified source code.
- ❖ **DLR3:** The system **shall** minimize manual intervention; automation is the default mode.

4.3. Functional Requirements (MoSCoW)

4.3.1. Must Have Requirements

DLR1 - US1: Activity Diagram Generation from Verified PlusCal

As a SanDisk firmware engineer, I want the system to automatically generate UML activity diagrams from my verified PlusCal specifications, so that I can visualize the control flow and decision logic of my algorithm without manually creating diagrams. An activity diagram is a diagram that shows the flow of activities or actions in a system or process.

Prototype:

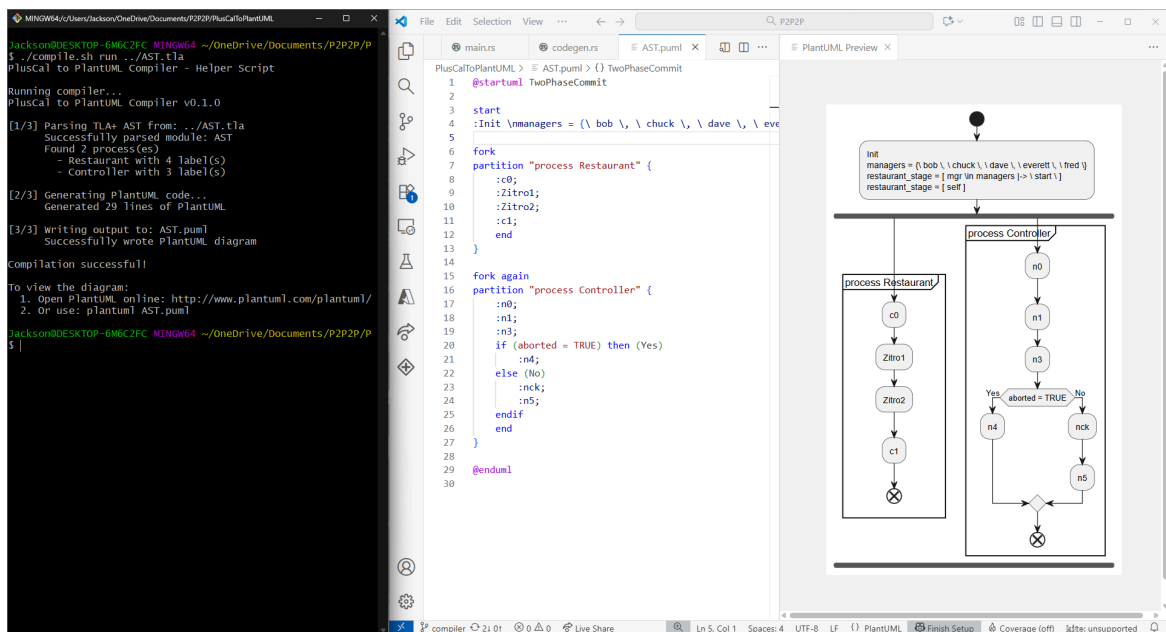


Figure 2: This diagram shows the program being run in the left terminal using the activity state diagram mode. This then generates the PlantUML file in the middle, which visually looks like the diagram on the right.

DLR1 - US2: State Diagram Generation from Verified PlusCal

As a SanDisk firmware engineer, I want the system to automatically generate UML state diagrams from my verified PlusCal specifications, so that I can visualize the different states of my system and the transitions between them. A state diagram (also called a state machine diagram) is a UML diagram that shows the different states of an object and the transitions between those states based on events.

Prototype:

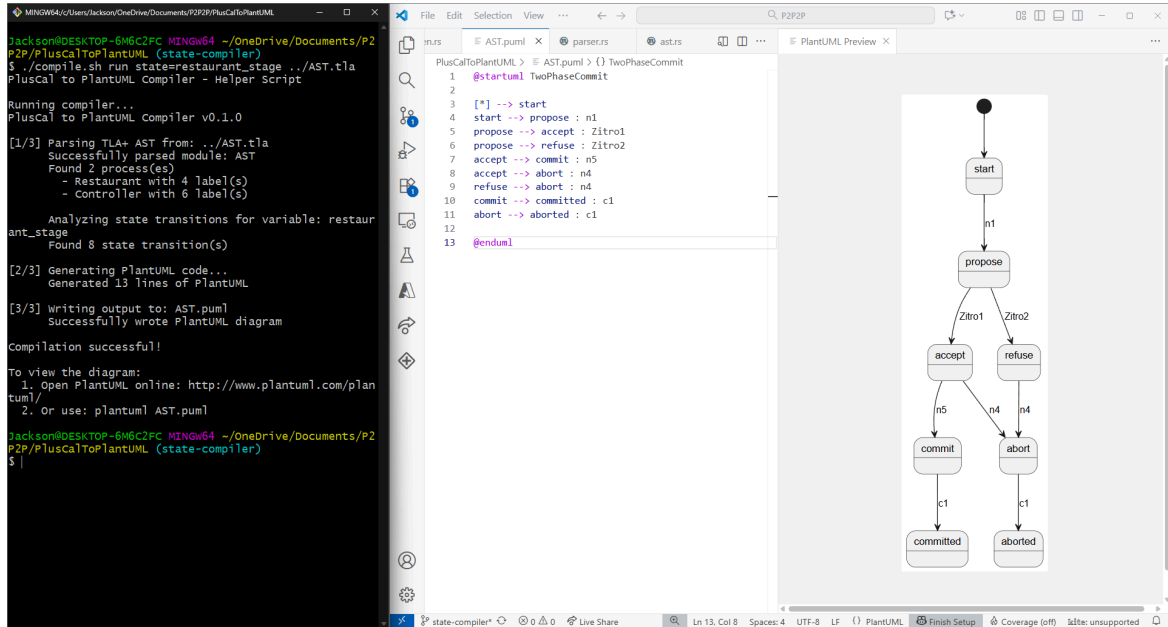


Figure 3: This diagram shows the program being run in the left terminal using the state diagram mode tracking the state “restaurant_stage” This then generates the PlantUML file in the middle, which visually looks like the diagram on the right.

DLR3 - US3: One-Click Documentation Generation in VSCode

As a SanDisk firmware engineer, I want a single button in my VSCode editor that generates complete documentation (diagrams + PDF) from my currently open PlusCal file, so that I can produce professional documentation with minimal effort and without switching tools.

Prototype:

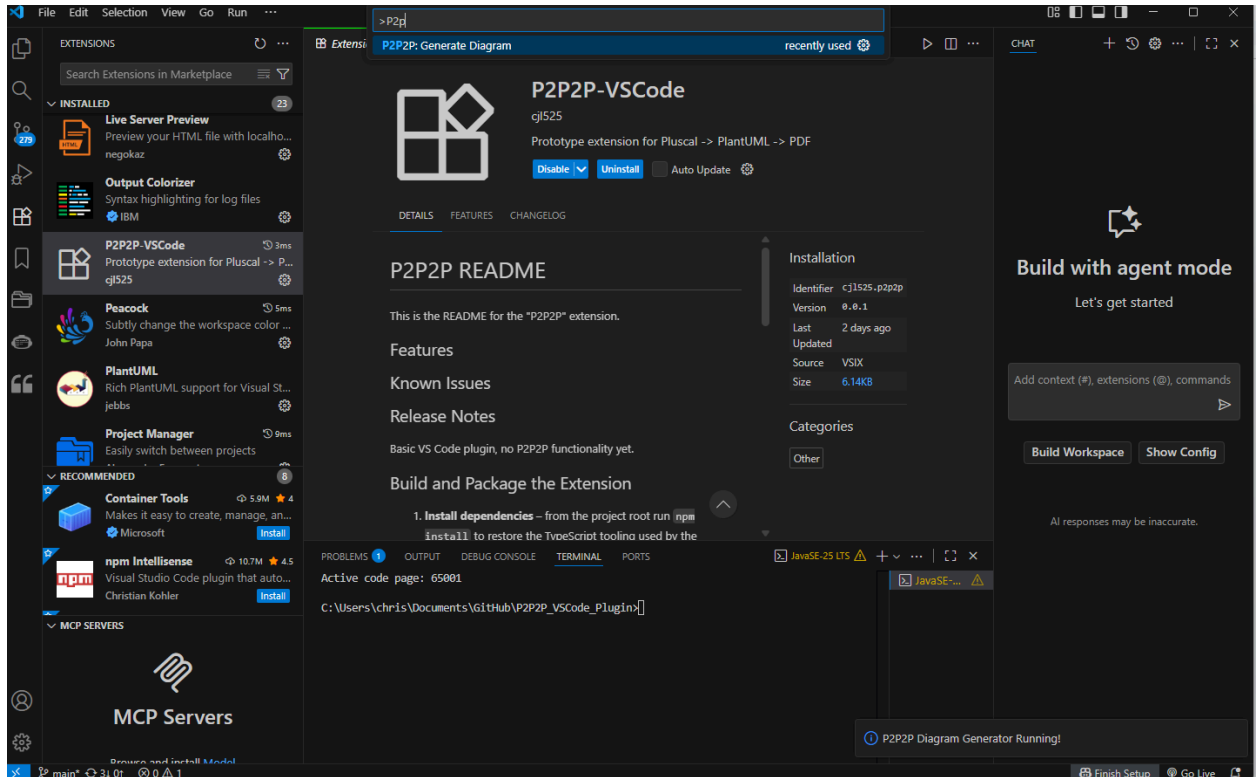


Figure 4: The VSCode interface showing the extensions section, with the VSCode command palette showing the command "P2P2P: Generate Diagram", and an output panel at the bottom showing generation progress. This prototype demonstrates seamless integration into the engineer's existing workflow.

DLR2 - US4: Automated PDF Compilation with Embedded Diagrams

As a SanDisk firmware engineer, I want the system to automatically compile my diagrams into a professional PDF synchronized with verified PlusCal code, so that I can quickly produce accurate, stakeholder-ready documentation without manual effort.

Prototype:

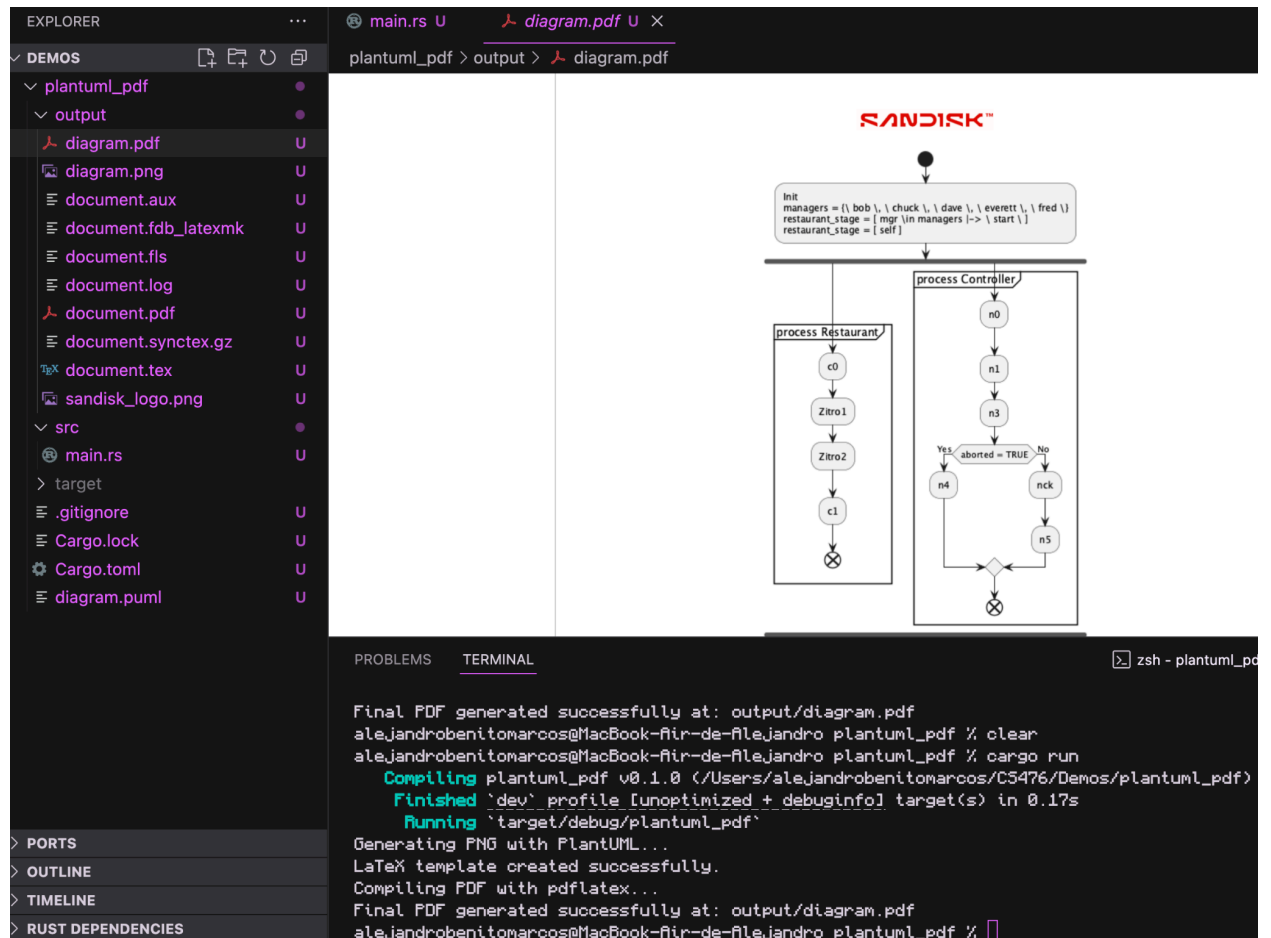


Figure 5: The VSCode workspace displaying the project structure of a PlantUML-to-PDF demo, with the generated diagram.pdf opened in the editor, showing an activity diagram featuring a SanDisk-branded header. The terminal at the bottom presents the successful build and PDF-generation logs.

4.3.2. Should Have Requirements

DLR3 - US5: Diagram Preview Before PDF Generation

As a SanDisk firmware engineer, I want to preview the generated diagrams in VSCode before committing to full PDF generation, so that I can verify the diagrams are correct and make adjustments to my specification if needed.

DLR3 - US6: Configurable Output Directory

As a SanDisk firmware engineer, I want to configure where the generated PDFs and diagrams are saved, so that I can organize documentation according to my project's directory structure or team conventions.

4.3.3. Could Have Requirements

DLR2 - US7: Configurable Diagram Themes

As a user, I want to be able to choose between different color themes and styles for generated diagrams, so that my documentation can match SanDisk's required themes, or different themes.

DLR1 - US8: Additional Diagram Types

As a developer, I want the option to generate other diagram types (such as flowcharts or sequence diagrams), so that I can visualize the system's behavior from different perspectives.

DLR2 - US9: Export to DOCX or HTML

As a user, I want to export my generated documentation in formats like DOCX or HTML, so that I can easily integrate it into company wikis or documentation systems.

DLR3 - US10: Integration with TLA+ VSCode Extension

As a developer, I want the system to operate as an add-on to the existing TLA+ VSCode extension, so that I can use both tools together without needing to switch environments.

4.3.4. Won't Have Requirements

DLR3 - US11: Full Web-Based Interface

As a user, I want to access the documentation generator through a web browser, so that I can use it without installing VSCode.

DLR3 - US12: Custom Diagram Editing Tools

As a user, I want to manually rearrange or edit diagram nodes inside the VSCode preview window, so that I can fine-tune the diagram's layout.

DLR3 - US13: Mobile Integration

As a user, I want to be able to use the P2P2P tool on a mobile device, so that I can create documents on my phone.

4.4. Non-Functional Requirements

4.4.1 Introduction

For this project, there are many requirements that don't describe how the system should function or be capable of doing, but instead describe constraints on measuring success based on those requirements. These are described as the non-functional requirements of the system, which tell us how performant the system should be, how readily the system can be adapted to technology changes, and how easy the system is to learn and integrate into existing work flows among other specific measurements we have for success listed below.

4.4.2 Performance Requirements

1. Runtime and Throughput:
 - a. **(US1 - US4)** - The system **shall** generate both the diagram and PDF output for an average-sized model in under 60 seconds, measured from the start of the generation process to the file being saved as output.
 - b. **(US1 - US2)** - The system **shall** process the Abstract Syntax Tree (AST) generated from the TLA+ specification at a rate of

at least 1,000 lines per second to determine the required diagram elements.

- c. **(US1 - US4)** - The system **shall** successfully handle large models containing up to 2,000 lines of code with a response time that scales linearly, not exceeding 120 seconds for that given file length.

2. Resource Consumption:

- a. **(US1, US2, US4)** - The program's peak RAM utilization during the generation of the largest expected model of 2,000 lines **shall not** exceed 1 GB of system memory. This gives plenty of storage to allow for deeply nested and diverging AST files and the overhead that will be required for storing the raw file information, and the processed information of that file during runtime.

4.4.3 Reliability Requirements

1. Accuracy:

- a. **(US1 - US2)** - The system **shall** generate a diagram that is 100% accurate in its representation of the TLA+ specification's structure, covering every edge-case parsing rule defined within the official TLA+ open-source repository specifications.
- b. **(US3 - US4)** - For any given TLA+ specification input, the generated output, diagram structure and PDF content, **shall** be identical across all executions and the process should be deterministic.

2. Error Handling:

- a. **(US1 - US4)** - The system **shall not** begin the parsing process if any of the following happen prior to the main P2P2P program is run:
 - During the process of generating the AST file from a PlusCal specification, an error or flag is given.
 - Translating PlusCal to TLA+ produces an error or flag.
 - The model does not pass the TLC

4.4.4 Maintainability Requirements

1. Updating:
 - a. **(US1 - US4)** - The system's codebase **shall** be structured to allow changes made to the underlying TLA+ Specification parsing logic, whether updating to a new TLA+ version or supporting new language features, to be incorporated into the generation functionality with an estimated effort of less than 2 weeks per major TLA+ language update.
2. Coding Standards:
 - a. **(US1 - US13)** - The project **shall** adhere to the standard Rust style guidelines by utilizing existing linting and formatting tools provided by The Rust Foundation, including rustfmt and clippy, to maintain a consistent style in written code and a static code analysis score of 95% or higher for warning avoidance.
3. Testability:
 - a. **(US1 - US2)** - The core parsing and diagram generation logic **shall** be covered by unit tests achieving a minimum of 75% line coverage.

4.4.5 Usability Requirements

1. Learnability:
 - a. **(US1 - US4)** - A first-time user **shall** be able to successfully generate their first diagram by following the provided instructions in under 5 minutes after the installation is complete.
2. Interface/Feedback:
 - a. **(US3 - US4)** - Any user interface elements or command-line feedback **shall** be provided to the user's action within 0.5 seconds.

4.5. Environmental Requirements

4.5.1 Introduction

The development of the PlusCal-to-PlantUML-to-PDF (P2P2P) documentation generator is subject to a few restraints given to us by the client, the target development ecosystem, and the technologies selected for integration. These requirements tell us the environment in which the system must operate successfully and the technologies it must use.

4.5.2 Hardware Constraints

1. The system is designed for use on standard engineering workstations.
2. A minimum of 8GB of RAM and a multi-core CPU for efficient diagram and document generation.
3. The system is not intended for mobile or tablet devices.

4.5.3 Software Constraints

1. The backend must be implemented in Rust, chosen for its safety guarantees and high-performance memory handling.
2. The system must use PlantUML to render UML activity and state diagrams from intermediate representations.
3. The system must depend on LaTeX to compile final PDF documentation.
4. The system must operate within the Visual Studio Code (VSCode) environment as an extension.
5. The system must be compatible with Git for source control and integration with SanDisk's internal workflows.

4.5.4 Operating System Constraints

1. The tool must run on Windows 11, macOS, and Linux.
 - a. Priority in order of Windows 11, then macOS, then Linux.
2. All scripts and config paths must be implemented using relative file paths to ensure cross-platform compatibility.

4.5.5 Integration Constraints

1. The system must integrate smoothly with existing TLA+ development workflows, supporting both .tla and .cfg files.
2. The tool must generate diagrams using PlantUML syntax conventions to ensure accurate rendering without any user modification.

4.5.6 Compliance Constraints

1. All code and documentation produced by the system must follow SanDisk's internal confidentiality and data handling policies.
2. The system must not store or transmit user data outside the local environment.

5. Potential Risks

5.1. Introduction

While the P2P2P system promises to streamline the workflow of engineers at SanDisk by automating diagram generation from verified PlusCal specifications, several risks could affect the success of this project. These risks are particularly relevant because errors or delays in this system could directly impact the accuracy of design documentation, the efficiency of engineering workflows, and ultimately the quality of SanDisk products. The relevance of a risk is determined both by its likelihood of occurrence and the potential severity of its impact on the system and its stakeholders.

5.2. Incorrect Diagram Generation

- ❖ **Description:** If the P2P2P tool produces diagrams that do not accurately reflect the verified PlusCal code, stakeholders may base decisions on incorrect designs.
- ❖ **Likelihood:** Medium. Parsing and converting formal code to PlantUML is complex but feasible with Rust.
- ❖ **Impact:** High. Inaccurate diagrams could lead to flawed design reviews, wasted engineering effort, and potential downstream product errors.
- ❖ **Mitigation Strategy:** Extensive unit and integration testing of the Rust parser, verification of diagram outputs against PlusCal/TLC outputs, incremental rollout with sample projects before full deployment.

5.3. Failure in Formal Verification Integration

- ❖ **Description:** Errors in detecting TLC verification status (e.g., misreading command-line output) could trigger diagram generation for unverified models.
- ❖ **Likelihood:** Low to Medium. Command-line parsing is straightforward but depends on TLC output format stability.
- ❖ **Impact:** High. Could produce diagrams for incorrect or incomplete models, undermining the system's reliability.
- ❖ **Mitigation Strategy:** Monitor TLC output format for changes, implement automated tests for the verification parsing module, provide error handling and alerts for failed verification.

5.4. VSCode Extension or Workflow Failures

- **Description:** Integration of the P2P2P pipeline into VSCode may fail or behave inconsistently across platforms (Windows, macOS, Linux).
- **Likelihood:** Medium. Cross-platform IDE extensions often face configuration or compatibility issues.
- **Impact:** Medium. Could delay engineer adoption and slow workflow efficiency improvements.
- **Mitigation Strategy:** Thorough cross-platform testing; provide fallback CLI option; clear user documentation and troubleshooting guides.

5.5. Build or LaTeX PDF Compilation Issues

- **Description:** LaTeX compilation errors or mismatched PlantUML inclusion could prevent reliable PDF generation.
- **Likelihood:** Medium. LaTeX syntax and dependency issues are common, especially for automated pipelines.
- **Impact:** Medium. Delays in final documentation could impact review timelines and stakeholder confidence.
- **Mitigation Strategy:** Automate build validation, embed test diagrams in LaTeX templates, maintain a CI/CD-like process for PDF generation during development.

5.6. Stakeholder Adoption Risk

- **Description:** Engineers accustomed to manual diagramming may resist adopting the automated workflow.
- **Likelihood:** Medium. User behavior is difficult to predict.
- **Impact:** Medium. Partial adoption could reduce the benefits of the tool and slow formal methods adoption.
- **Mitigation Strategy:** Incorporate feedback loops during pilot testing, provide documentation and demonstrations, ensure the tool saves demonstrable time and reduces errors.

The team recognizes that a proactive approach that identifies and mitigates risks minimizes the likelihood and impact of these risks. Regular testing, error handling, scaling rollout, and communication with stakeholders are components of this proactive approach in order to ensure the P2P2P system has accurate, reliable, and efficient outputs. Although some risks cannot be eliminated, understanding their impact ensures planning contingencies, maintaining the integrity of design

documents, and establishing the process for supporting the instantiation of formal method adoption at SanDisk.

6. Project Plan

EXTEND PROJECT PLAN TO FULLY EXPLAIN FUTURE PLANS BEYOND WHAT IS CURRENTLY SHOWN, EXPLAINING MORE THAT THE CURRENTLY IMPLEMENTED FEATURES IN THE PROTOTYPES DO NOT SHOW THE FULL SCOPE OF THE PROJECT AND THERE IS STILL A LOT OF GRUNT WORK FOR FULL FUNCTIONALITY AND SUPPORTING MORE SYSTEMS AND KEYWORDS.

6.1 Introduction

At the current stage of the project, there are a handful of individual prototypes that work at a much lower scope than is necessary for the project, but are functional to the extent necessary to display feasibility and potential future capabilities. To move forward in the project, the focus of the team is going to be on both improving individual prototypes, by looking at resources such as the AST documentation for TLA+, moving towards combining the individual separate tools that have been developed so far into one cohesive unit, and fitting the output to specific style guidelines of the client, creating the final P2P2P tool.

6.2 Milestones

In order to track the progress of these goals moving forward in the project, we have identified various milestones that must be reached:

- **Broad range support for bare bones activity diagrams.**
 - This milestone will be one of the first completed due to its need for future development, expected to be finished by early January 2026.
- **Broad range support for bare bones state diagrams.**
 - Same as the activity diagram, supporting this feature is integral to future development and is expected to be finished in early January 2026.
- **Integration of full work pipeline, the ability to go from PlusCal to PlantUML to PDF seamlessly through the VS Code Extension.**
 - This will be a step towards merging existing features into one cohesive tool once the individual pieces are in an acceptable state, so integration is expected to be done by early February 2026.
- **Support for the full range of AST keywords.**
 - This is an ongoing milestone that will be one of the last to be done, because once this milestone is done, the project is nearly, if not, complete. Expected completion is the middle of March 2026.
- **Full formatting to SanDisk document Standards.**
 - To fully incorporate our tool into the work flow of our client, we will need to make sure the output PDF and Diagram strictly adheres to

standards that are consistent with SanDisk documentation. This can be completed before completion of all the tools, so expected completion is middle of February 2026.

Project plan GANTT chart available in Appendix A.

7. Conclusion

The P2P2P (PlusCal to PlantUML to PDF) project represents a strategic step toward improving the efficiency, accuracy, and reliability of SanDisk's design verification and documentation workflow. In a company where the correctness of firmware designs directly impacts millions of devices, the problem of disconnected manual documentation has significant implications for productivity, quality, and formal adoption. By automating the creation of synchronized diagrams and verified documentation, P2P2P directly addresses these limitations and supports SanDisk's broader initiative to embed formal methods into daily engineering practice.

This Requirements Document outlined the motivations behind the project, the specific problems in the current workflow, and a clear technical vision for the solution. The proposed system integrates verified PlusCal specifications with PlantUML diagram generation, automated LaTeX PDF compilation, and seamless Visual Studio Code integration, delivering a one-click workflow from model verification to professional documentation. The defined functional and non-functional requirements, environmental constraints, and risk mitigation strategies collectively establish a realistic path for development and deployment.

To date, the team has validated key technological components through prototype work in parsing, diagram generation, and PDF automation, demonstrating the feasibility of the full pipeline. The next steps include unifying these modules into a single VSCode extension and refining the interface for ease of use and compliance with SanDisk's documentation standards.

With a solid technical foundation, a clear implementation roadmap, and continuous collaboration with our sponsor, the development team is confident that P2P2P will deliver a reliable, scalable, and transformative tool for SanDisk's engineering teams. The project stands to not only enhance documentation quality and efficiency but also reinforce SanDisk's leadership in adopting formal, verifiable design practices across the data storage industry.

8. Glossary and Appendices

Term / Acronym	Definition
AST (Abstract Syntax Tree)	A tree-structured way to represent source code that shows its grammatical structure. Used by the P2P2P system to translate PlusCal into diagram elements.
CI/CD (Continuous Integration / Continuous Deployment)	A software practice automating building, testing, and deployment for faster, and more reliable releases.
Formal Verification	Mathematical process proving a system behaves according to its specification.
LaTeX	Typesetting system used to produce professional PDFs with consistent formatting.
PlantUML	Tool that converts text-based descriptions into UML diagrams.
PlusCal	A pseudocode language automatically translated into TLA+, allowing algorithmic specifications for verification.
P2P2P	<i>PlusCal</i> $\rightarrow$ <i>PlantUML</i> $\rightarrow$ <i>PDF</i> , the team's automated documentation system.
Rust	Programming language emphasizing performance and memory safety, used for the backend parser.
SanDisk	Project sponsor and subsidiary of Western Digital specializing in flash storage.
TLA+ (Temporal Logic of Actions)	Formal specification language for describing concurrent system behavior.
TLC (TLA+ Model Checker)	Tool verifying that a TLA+ specification satisfies its properties before diagram generation.
UML (Unified Modeling Language)	Standard visual modeling language for describing system behavior and structure.
VSCoDe (Visual Studio Code)	Widely used code editor which will be used to host the P2P2P extension.

[illegible]

[illegible][illegible]